

YANG Libraries Feature Comparison

A Feature-Based Gap Analysis of libyang, Yangkit and ODL Yang Tools

Vivekananda Boudia — vivekananda.boudia@insa-lyon.fr

Pierre Francois — pierre.francois@insa-lyon.fr

Maxence Younsi — maxence.younsi@insa-lyon.fr

Version 1.0

[Abstract](#)

[1. Introduction](#)

[2. Research Objective and Scope](#)

[2.1 Research Objective](#)

[2.2 Libraries Included in the Detailed Analysis](#)

[2.3 Libraries Discarded from the Detailed Comparison](#)

[2.4 Functional Scope](#)

[3. Methodology](#)

[3.1 Feature-Based Evaluation Strategy](#)

[3.2 Support-Level Terminology](#)

[3.3 Test Artifacts and Repository Organization](#)

[3.4 Use of Code Excerpts](#)

[3.5 Comparative Reading Principles](#)

[4. Schema Handling](#)

[4.1 Module Loading and Validation](#)

[4.2 Module Inspection](#)

[5. Data Validation](#)

[5.1 JSON-Encoding Validation](#)

[5.2 Mandatory and Must Constraint Validation](#)

[5.3 CBOR-Encoding Validation](#)

[6. Anydata Handling](#)

[6.1 Role of Anydata in the Analysis](#)

[6.2 libyang Behavior](#)

[6.3 Yangkit Behavior](#)

[6.4 ODL Yang Tools Behavior](#)

[7. YANG Structures and Anydata-Structure Interaction](#)

[7.1 YANG Structures](#)

[7.2 Anydata Inside Structures](#)

[7.3 Structures Inside Anydata](#)

[8. XPath-Related Extraction and Subtree Validation](#)

[8.1 Schema Extraction](#)

[8.2 Data Extraction](#)

[8.3 Subtree Validation](#)

[9. YANG-to-YANG Transformation Enablers](#)

[9.1 Transformation Objective](#)

[9.2 libyang Behavior](#)

[9.3 Yangkit and ODL Yang Tools Behavior](#)

[10. YANG Library Loading](#)

[11. Yang Push, Telemetry and Notification Envelope Validation](#)

[11.1 Yang Push Validation](#)

[11.2 Telemetry Message Validation](#)

[11.3 Notification Envelope Validation](#)

[12. Schema Comparison](#)

[13. Cross-Feature Synthesis](#)

[13.1 Interpretation by Workflow Type](#)

[13.2 Key Technical Differentiators](#)

[14. Conclusion](#)

[15. References and Sources](#)

Abstract

This report presents a feature-based gap analysis of three YANG libraries: libyang, Yangkit and ODL Yang Tools. The analysis draws on the functional coverage documented by the study and on the associated test repository used to exercise representative schema, data and message-validation scenarios. Its objective is to provide a structured comparison of their observed behavior across a selected set of YANG-related features.

The comparison covers schema loading and module inspection, JSON and CBOR data validation, anydata handling, YANG structures, interaction between anydata and structures, XPath extraction, subtree validation, YANG-to-YANG data transformation, YANG Library loading, Yang Push message validation, telemetry message validation, notification envelope validation and schema comparison feasibility.

1. Introduction

YANG-based tooling plays an important role in software systems that need to load, inspect, validate, and manipulate data modeled according to YANG schemas. Several libraries exist for these purposes, and most of them provide core capabilities such as parsing YANG modules, building schema contexts, and validating data instances. However, practical usage of YANG libraries is not limited to basic module parsing. It often involves more advanced operations, including validation of encoded data, subtree extraction, support for telemetry-oriented messages, and direct manipulation of data trees.

This broader usage raises an important issue: the YANG ecosystem continues to evolve. New constructs, extensions, encodings, and operational use cases are introduced over time, but library support does not always evolve at the same pace. As a result, a library may provide mature support for traditional schema and data validation tasks while remaining incomplete or inconsistent for newer or more specialized features such as anydata, YANG structures, notification envelopes, or telemetry messages.

The present study investigates this gap by comparing the behavior of several YANG tools against a set of features. The detailed analysis focuses on libyang, Yangkit, and ODL Yang Tools. Rather than evaluating these libraries only as parsers, the study examines how they behave when applied to representative use cases involving schema handling, data validation, advanced YANG constructs, subtree processing, and telemetry-oriented message validation.

The analysis is organized by functional area. For each feature, the report describes the purpose of the capability, compares the observed behavior of the libraries, and discusses the technical implications of the differences identified through the tests.

2. Research Objective and Scope

2.1 Research Objective

The objective of this work is to analyze how selected YANG libraries behave across a defined set of features and to identify where their capabilities converge or diverge. The analysis is feature-based: the same conceptual capability is considered across the libraries, and support is characterized using a common terminology.

The comparison concerned functional behavior: whether a feature is available, whether it can be obtained by composing lower-level APIs, whether the tested behavior is partial, and whether a scenario cannot be evaluated because a prerequisite feature is missing.

2.2 Libraries Included in the Detailed Analysis

The detailed analysis covers three libraries. They differ in implementation language, ecosystem and API design. These differences matter because they affect both the form of the test code and the way validation steps are expressed.

Table 1. Libraries included in the feature-based comparison

Library	Language	Evaluated version	Role in the analysis
libyang	C	5.4.9	Evaluated as a mature C-based implementation with command-line support through yanglint and programmatic APIs for context and data-tree manipulation.
Yangkit	Java	1.6.0	Evaluated as a Java toolkit supporting schema parsing, data validation, JSON/CBOR-related workflows and structure-oriented scenarios.
ODL Yang Tools	Java	16.0.0-SNAPSHOT / 16.0.0	Evaluated as a Java library with schema and normalized-node APIs, but with some advanced scenarios constrained by structure support.

2.3 Libraries Discarded from the Detailed Comparison

Several libraries were considered at the beginning of the work but were not retained for the detailed feature-by-feature comparison. Some candidates relied on external validation workflows, some were considered too old and not maintained, and some were wrappers around libyang without providing an independent feature set for the analysis.

Table 2. Libraries considered but discarded early in the project

Library or family	Reason for exclusion from detailed analysis
pyang	Relies on transformation to DSDL and XML validators.
yangson	Described as an older YANG library; using it would require starting from a comparatively distant baseline.
Verizon YANG Python wrapper	Depends on libyang and adds specific features rather than providing an independent comparison target.
yang3 / yang4	Rust wrappers depending on libyang; no additional feature coverage beyond connecting to libyang is identified in the test results.
yang-rs	Rust wrapper depending on libyang; not retained as a distinct functional baseline.

2.4 Functional Scope

The feature scope covers schema-related operations, encoded data validation, advanced payload constructs, extraction and subtree-oriented validation, transformation capabilities, and telemetry or notification-oriented messages. The purpose of this scope is to reflect realistic usage in which YANG libraries are expected to process not only static modules, but also data instances and messages that depend on multiple schemas.

Table 3. Functional scope of the gap analysis

Feature family	Capabilities considered
Schema	Module loading and validation; module inspection.
Data	Validation of JSON-encoded data instances; validation of CBOR-encoded data instances.
Anydata	Validation of anydata.
Structures	Validation of YANG structures.
Anydata and structures	Validation of anydata inside structures and validation of structures inside anydata.
XPath and subtree validation	Schema extraction, data extraction, anydata-related extraction, structure-related extraction and validation of subscription subtrees against original schemas.
YANG-to-YANG transformation	Modification of data instances without going through an intermediate encoding.
YANG Library	Loading a list of modules and enabling features from YANG Library data.
Yang Push and telemetry	Validation of Yang Push messages, telemetry messages and notification envelope messages.
Schema comparison	Feasibility of implementing the YANG schema comparison draft or generating comparison data.

3. Methodology

3.1 Feature-Based Evaluation Strategy

The analysis follows a feature-based methodology. For example, for JSON data validation is whether the library can validate a JSON data instance against a schema and reject invalid data. For anydata, the question is more nuanced: the analysis considers whether primitive payloads, objects without schema, and objects with schema are handled according to the expected validation model.

This approach is appropriate because the libraries expose different APIs and use different internal abstractions. A direct one-to-one API comparison would therefore be misleading. Instead, the analysis focuses on whether the same functional outcome can be achieved, either directly through a native API or indirectly by assembling existing calls.

3.2 Support-Level Terminology

A consistent vocabulary is used to describe the result of each feature evaluation.

Table 4. Support-level terminology used in the report

Support level	Meaning in this report
Supported	The feature is available and the tested scenario succeeds, either through a direct API or through a clear sequence of library calls.
Partially supported	The feature is available for some tested scenarios, but the behavior is incomplete, constrained, or inconsistent with the expected result in at least one important case.
Not supported	The feature is not available in the evaluated version or the tested scenario cannot be achieved with the available mechanisms.
Not applicable	The scenario depends on another feature that is not supported, so the specific test cannot be meaningfully executed.

3.3 Test Artifacts and Repository Organization

The test repository associated with the analysis is organized around the same functional areas. It contains YANG modules, JSON data instances, Java tests for Yangkit and ODL Yang Tools, and related folders for data-validation, anydata, structures, XPath, subtree validation, YANG Library, Yang Push, telemetry, notification and transformation scenarios.

When a code excerpt is used, the corresponding repository path is indicated below the excerpt to make the observation traceable without overloading the main text.

3.4 Use of Code Excerpts

Selected code excerpts are used as technical evidence for behavioral differences, validation boundaries and implementation constraints. They are kept short and limited to the relevant test logic.

4. Schema Handling

4.1 Module Loading and Validation

Schema handling is the first requirement for any YANG processing workflow. Before data instances can be validated, the library must be able to load YANG modules, resolve dependencies and reject invalid module definitions. The three evaluated libraries support module loading and validation, but validation is triggered at different stages.

In libyang, module validation is performed automatically when a module is loaded into the context. This creates a direct relationship between context construction and schema correctness: loading a module is already a validation operation. The corresponding test example illustrates this behavior by loading two modules.

```
/* Load modules */
static void test_load_modules(void** state) {
    assert_non_null(ly_ctx_load_module(ctx, "schema-test",
    NULL, NULL));
    assert_non_null(ly_ctx_load_module(ctx, "nmda-test",
    NULL, NULL));
}
```

Source: libyang module-loading evaluation test.

ODL Yang Tools validates modules through the construction of the effective model. In the tested workflow, `.buildEffectiveModel()` is the operation that raises an error when the module is not valid. Yangkit separates the process into two stages: `.parse()` checks YANG syntax, while `.validate()` checks internal consistency, including whether referenced identities can be resolved.

Table 5. Comparison of module loading and validation behavior

Feature	libyang	Yangkit	ODL Yang Tools
Module loading	Supported. Modules are loaded into a libyang context.	Supported. Modules are parsed through the Yangkit parser workflow.	Supported. Modules are loaded and processed into an effective model.
Module validation	Supported. Validation is automatically performed when a module is loaded.	Supported. Syntax and consistency checks are represented as separate parse and validate steps.	Supported. Invalid modules lead to errors during effective model construction.
Interpretation	Validation is closely tied to context loading.	The split between parsing and validation makes the validation stages explicit.	The effective-model step centralizes semantic validation.

4.2 Module Inspection

Module inspection concerns the ability to retrieve information such as imports, versions and dependencies, potentially without fully integrating the module into a global schema context. This is relevant when tooling needs to analyze module metadata before deciding how to build a complete schema context.

The inspection tests indicate that libyang does not simply expose an abstract syntax tree independently of a context. A temporary context can be used, but the workflow remains context-oriented. Yangkit and ODL Yang Tools are more flexible in this respect: both can inspect a module either in a context or without a context. Yangkit requires initializing its own parser for this usage.

Table 6. Comparison of module inspection capabilities

Feature	libyang	Yangkit	ODL Yang Tools
Module inspection	Partially supported. Inspection is possible through a context, but the library does not simply expose an AST independently of loading.	Supported. A module can be inspected in or outside a context, with parser initialization.	Supported. A module can be inspected in or outside a context.
Analytical impact	The context requirement may constrain pre-loading inspection workflows.	Provides a clearer path for standalone module analysis.	Provides a clearer path for standalone module analysis.

5. Data Validation

5.1 JSON-Encoding Validation

JSON data validation is a core feature because many YANG-modeled data instances use a JSON representation. The common evaluation question is whether the library can parse a JSON instance, associate it with the corresponding YANG schema, and reject data that violates the model.

The three libraries support JSON validation. Libyang validates JSON data instances through parsing functions that wrap `lyd_parse_data_mem` and reject invalid data. Yangkit and ODL Yang Tools also reject invalid JSON instances, but their validation workflows are expressed through Java test helpers that load schema files and data files. The Yangkit tests show both valid data and several invalid cases: invalid string validation, numeric validation for a hostname, out-of-bounds port values, missing mandatory data and invalid `must` constraints.


```

@Test
void testValidJsonValidation() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadValidYangDataDoc("../yang/data-validation/data-validation-test.yang",
    "../data/data-validation/data-validation-test-valid.json");
}

@Test
void testInvalidJsonValidation() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadInvalidYangDataDocParseError("../yang/data-validation/data-validation-test.yang",
    "../data/data-validation/data-validation-test-invalid-string-validation.json");
}

@Test
void testMissingMandatoryValidation() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadInvalidYangDataDocValidateError("../yang/mandatory-validation/mandatory-test.yang",
    "../data/mandatory-validation/missing.json");
}

@Test
void testInvalidMustValidation() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadInvalidYangDataDocValidateError("../yang/must-validation/must-test.yang",
    "../data/must-validation/invalid-must.json");
}

```

Source: yangkit/src/test/java/DataValidationJsonEncodingTest.java

5.2 Mandatory and Must Constraint Validation

The validation tests distinguish schema matching from deeper constraint validation. A data instance may be structurally associated with a YANG schema while still violating constraints such as `mandatory` or `must`. ODL Yang Tools behaves similarly to Yangkit in this respect: first the data is checked against a schema, then an additional validation step checks constraints.

This distinction is important because a library may appear to parse data successfully while still requiring a second call to enforce full YANG constraints. For research or production usage, the validation workflow must therefore be read carefully: a successful parse is not necessarily equivalent to a complete validation result unless the relevant constraint-checking step has been executed.

5.3 CBOR-Encoding Validation

CBOR support is more uneven than JSON support. The research results indicate that libyang had an open pull request related to CBOR and that the pull request was still open at the time of the analysis. ODL Yang Tools uses GSON for JSON parsing, which does not support CBOR. Yangkit relies on Jackson to parse CBOR and convert it into the same Java representation as JSON, making the validation process identical for both formats. This does not cover CBOR SIDs.

Table 7. Comparison of data-validation capabilities

Feature	libyang	Yangkit	ODL Yang Tools
JSON data validation	Supported. Valid data is parsed and invalid data is rejected.	Supported. Valid and invalid JSON data instances are covered by tests.	Supported. JSON data instances are validated against the schema.
Mandatory constraints	Supported as part of data validation.	Supported through an additional validation-error path.	Supported through additional validation after schema matching.
Must constraints	Supported as part of data validation.	Supported through explicit valid and invalid must tests.	Supported through validation workflow described in the test results.

Feature	libyang	Yangkit	ODL Yang Tools
CBOR data validation	Partially supported / pending in the analysis due to an open PR.	Supported for self-contained CBOR through Jackson conversion to JSON; CBOR SIDs are not covered.	Not supported in the tested workflow because the JSON parsing stack uses GSON and CBOR is not supported there.

6. Anydata Handling

6.1 Role of Anydata in the Analysis

The anydata construct is a central part of the gap analysis because it tests how far each library can validate content whose schema may be known only through the payload itself or through additional context. The analysis considers several cases: primitive anydata values, objects that have a corresponding YANG schema, objects that do not have a corresponding schema, arrays, and null values.

The practical difficulty is that anydata validation must balance flexibility and strictness. A library that accepts arbitrary content may be too permissive for the expected Netmod usage, while a library that cannot resolve schema-backed payloads cannot validate advanced telemetry or envelope structures properly.

6.2 libyang Behavior

The libyang test case demonstrates strict validation for anydata nodes. It loads the anydata module and the referenced example module before parsing the JSON with strict anydata behavior. This indicates that libyang can validate the content behind an anydata node when the relevant schemas are available.

```
/* anydata validation */
static void test_anydata_validation(void** state) {
    ly_ctx_load_module(ctx, "anydata-test",
        NULL, NULL);
    ly_ctx_load_module(ctx, "example", NULL,
        NULL);
    struct lyd_node* tree = parse_json(ctx,
        "../data/anydata.json",
        LYD_PARSE_ANYDATA_STRICT,
        LYD_VALIDATE_PRESENT);
    assert_non_null(tree);
    lyd_free_all(tree);
}
```

Source: libyang anydata-validation evaluation test.

6.3 Yangkit Behavior

Yangkit provides useful support for anydata validation, but the behavior is partial in the tested cases. The tests show that objects with a YANG schema are validated against the schema and that objects without a YANG schema raise an `Unrecognized Element` error. We have strict validation like libyang. However, primitive values such as numbers or strings do not raise an error in the tested scenario, even though the expected Netmod behavior is for the payload to be an object.

```

@Test
void testObjectWithSchemaAnydata() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadValidYangDataDoc("../yang/anydata",
    "../data/anydata/object-with-schema-anydata.json");
}

@Test
void testObjectWithoutSchemaAnydata() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadInvalidYangDataDocParseError("../yang/anydata",
    "../data/anydata/object-without-schema-anydata.json");
}

@Test
void testEmptyArrayAnydata() throws Exception {
    YangkitUtils.loadInvalidYangDataDocParseError("../yang/anydata",
    "../data/anydata/empty-array-anydata.json");
}

```

Source: yangkit/src/test/java/AnydataValidationTest.java

6.4 ODL Yang Tools Behavior

ODL Yang Tools shows the most problematic anydata behavior in the tested cases. Primitive values raise an `Object model not set` error, which is consistent with rejecting primitive anydata payloads. Objects without schema also raise an `unknown element` error, which is expected. However, objects with schema still raise an `unknown element` error in the tested scenario, even though schema-backed object validation is the desired behavior.

The implementation-level analysis explains this result as follows: `EmptyAnydataEffectiveStatement` is not a `DataContainer`, `potentialChildNode` remains null, and `childChoices` is empty. This makes the issue analytically significant because it is not merely a test-data mismatch; it reflects the way the anydata effective statement is represented during child lookup.

```

@Test
void testObjectWithSchemaAnydata() throws Exception {
    YangToolsUtils.loadValidNormalizedNode(
    "../yang/anydata",
    "../data/anydata/object-with-schema-anydata.json"
    );
}

@Test
void testObjectWithoutSchemaAnydata() throws Exception {
    YangToolsUtils.loadInvalidNormalizedNodeParseError(
    "../yang/anydata",
    "../data/anydata/object-without-schema-anydata.json"
    );
}

```

Source: yangtools/src/test/java/AnydataValidationTest.java

Table 8. Comparison of anydata validation behavior

Scenario	libyang	Yangkit	ODL Yang Tools
Primitive anydata payload	Supported strict rejection in the strict validation model.	Partially supported. Primitive values should not be allowed, but no error is thrown in the tested case.	Supported rejection. Primitive value throws an error.
Object without schema	Supported rejection under strict validation.	Supported rejection with `Unrecognized Element`.	Supported rejection with `unknown element`.
Object with schema	Supported validation when referenced schema is loaded.	Supported validation against schema.	Partially supported / unexpected behavior. The test scenario raises `unknown`

Scenario	libyang	Yangkit	ODL Yang Tools
			element` even with schema available.
Analytical impact	Provides the strictest behavior in the tested anydata scenario.	Usable for schema-backed object payloads but too permissive for primitive values.	Blocked for schema-backed object payloads in the observed behavior.

7. YANG Structures and Anydata-Structure Interaction

7.1 YANG Structures

YANG structures are treated separately because they directly affect subsequent telemetry and notification-envelope scenarios. A library may support ordinary module loading and JSON data validation while still being unable to process structure-based messages. The tests and evaluation results show a clear distinction among the libraries in this area.

Libyang has existing tests for structures in `libyang/tests/utests/extensions/test\_structure.c`, including tests for schema, parsing and invalid schema cases. Yangkit supports structures through the Scott Huang pull request associated with the implementation under test. ODL Yang Tools contains structure-related code, but structures are not parsed by the schema parser in the tested state.

```
@Test
void testValidJsonStructure() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadValidYangDataDoc("../yang/scotthuang-structure",
    "../data/structure/scotthuang_valid_structure_message.json");
}

@Test
void testMissingMandatoryJsonStructure() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadInvalidYangDataDocValidateError("../yang/scotthuang-structure",
    "../data/structure/scotthuang_invalid_structure_missing_mandatory.json");
}

@Test
void testInvalidStructureTimestampValue() throws Exception {
    YangkitUtils.loadInvalidYangDataDocParseError("../yang/scotthuang-structure",
    "../data/structure/scotthuang_invalid_structure_invalid_timestamp_value.json");
}
```

Source: yangkit/src/test/java/YangStructureTest.java

7.2 Anydata Inside Structures

The interaction between anydata and structures is a more advanced scenario. The libyang structure tests also cover anydata content inside a structure. Yangkit can validate an anydata node inside a structure using the same validation behavior as in standalone anydata. This means that the same strengths and weaknesses apply: schema-backed objects are validated, unknown objects are rejected, and primitive values are not rejected in the observed case even though they should not be allowed. ODL Yang Tools is not applicable in this scenario because both anydata and structures are insufficiently supported for the required validation test.

```

@Test
void testObjectWithSchemaAnydataInStructure() throws DocumentException, IOException, YangParserException
{
    YangkitUtils.loadValidYangDataDoc("../yang/anydata-in-structure",
    "../data/anydata-in-structure/object-with-schema-anydata-in-structure.json");
}

@Test
void testObjectWithoutSchemaAnydataStructure() throws DocumentException, IOException, YangParserException
{
    YangkitUtils.loadInvalidYangDataDocParseError("../yang/anydata-in-structure",
    "../data/anydata-in-structure/object-without-schema-anydata-in-structure.json");
}

@Test
void testStructureInAnydata() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadValidYangDataDoc("../yang/structure-in-anydata",
    "../data/structure-in-anydata/structure-in-anydata.json");
}

```

Source: yangkit/src/test/java/YangAnydataStructureTest.java

7.3 Structures Inside Anydata

The reverse scenario, validating a structure inside an anydata node, is also covered. Libyang can validate a structure data node inside anydata, as illustrated by the `yanglint` command used in the evaluation. Yangkit can also build data where the payload is defined as anydata and contains a YANG structure. ODL Yang Tools is not applicable in this scenario because both anydata and structures are insufficiently supported for the required validation test.

Table 9. Comparison of structure and anydata-structure scenarios

Feature	libyang	Yangkit	ODL Yang Tools
YANG structures	Supported. Existing libyang tests cover schema, parsing and invalid schemas.	Supported. Structure support is available in the tested Yangkit branch/version.	Not supported in the tested state. Structures exist in code but are not parsed by the schema parser.
Anydata inside structure	Supported. Existing structure tests also validate anydata content.	Partially supported. The same anydata behavior applies, including primitive-value permissiveness.	Not applicable because anydata is not fully supported and structures are not supported.
Structure inside anydata	Supported through yanglint and schema-backed validation.	Supported. A YANG structure can be placed inside an anydata payload and validated.	Not applicable because the required structure and anydata support is missing.
Analytical impact	Covers advanced structure scenarios required by message validation.	Covers advanced scenarios, with the anydata caveat inherited from standalone anydata.	Cannot be used to evaluate structure-dependent message scenarios in the tested state.

8. XPath-Related Extraction and Subtree Validation

8.1 Schema Extraction

Schema extraction refers to retrieving a sub-part of a YANG schema. The tests mark this capability as supported in libyang, while Yangkit and ODL Yang Tools are also able to retrieve part of a schema. This is an important prerequisite for workflows that need to isolate the schema corresponding to a subscription path or a subset of a larger model.

8.2 Data Extraction

Data extraction is more differentiated. libyang supports basic XPath and can retrieve a value behind anydata in the tested example. The corresponding test uses `lyd_find_xpath` with an XPath that crosses into an anydata payload and retrieves `routerX` as the expected hostname.

ODL Yang Tools does not use “real XPath” in the same sense. The tested approach relies on a path that identifies a data node and allows the code to access it. This difference is important because it affects portability: a workflow expressed as XPath may need to be adapted to the normalized-node path abstractions used by ODL Yang Tools.

Yangkit shows mixed behavior in the tests. XPath to a leaf value works, XPath to a container returns null, XPath inside anydata is shown as null in the tested case, and XPath in structures works. Other functions can still retrieve the Java object representing the container, so a null XPath result does not necessarily mean that the object is inaccessible through the library.

8.3 Subtree Validation

Subtree validation is the ability to validate data coming from a subscription using an XPath against the original schema. The tested workflow consists of loading the original schema, evaluating the XPath to obtain the corresponding schema node, creating a new context from that node, and validating the received data against that context. The evaluation marks this capability as supported for all three libraries.

Table 10. Comparison of XPath-related and subtree-validation capabilities

Feature	libyang	Yangkit	ODL Yang Tools
Schema extraction	Supported.	Supported.	Supported.
Data extraction	Supported for basic XPath and XPath behind anydata.	Partially supported. Leaf and structure-related cases work; container and anydata cases show null in the test results.	Partially supported. Provides path-based node access rather than real XPath as described in the test results.
Anydata XPath extraction	Supported in the demonstrated example.	Partially supported; XPath inside anydata is shown as null.	Not clearly supported as real XPath; constrained by anydata behavior.
XPath in structures	Supported in the relevant structure context.	Supported according to the test results.	Not applicable where structure support is missing.
Subtree validation	Supported.	Supported.	Supported.

9. YANG-to-YANG Transformation Enablers

9.1 Transformation Objective

The transformation feature evaluates whether a library can support code that transforms a YANG data instance into another YANG data instance without serializing to an intermediate encoding. This is a data-tree manipulation problem rather than a pure validation problem. It tests whether nodes can be created, added, updated, removed or reconnected within an existing model-aware tree.

9.2 libyang Behavior

Libyang provides many functions to create or modify an existing data tree. The evaluated operations include adding new nodes, reconnecting nodes from one tree to another, moving nodes from one list instance to another and removing nodes. This indicates broad support for direct data-tree manipulation.

9.3 Yangkit and ODL Yang Tools Behavior

For ODL Yang Tools, the tests distinguish three cases. If a node is defined in the module but absent from the data, it can be added. If the node is defined in the module and present in the data, it can be updated. If the node is not defined in the module, it cannot be added.

For Yangkit, the addition of a schema-defined node that is absent from the data is possible. However, the tests show that a node already defined in both the module and the data cannot be updated in the current behavior, although this behavior could be changed in the code. Like ODL Yang Tools, Yangkit cannot add a node that is not defined in the module.

```
@Test
void addValueTest() throws Exception {
    YangSchemaContext context = YangkitUtils.loadValidSchema("../yang/yang-transformation");
    YangDataDocument doc = YangkitUtils.loadValidYangDataDoc(context,
        "../data/yang-transformation/yang_transformation.json");
    ...
    var container = (ContainerDataImpl)
    doc.getDataChild(YangkitUtils.getIdentifier("urn:yang:transformation", "foo"));
    container.addChild(leafNode);
    ...
    assertEquals("3", value);
}

// cannot use yangkit to update data
@Test
void updateValueTest() throws Exception {
    YangSchemaContext context = YangkitUtils.loadValidSchema("../yang/yang-transformation");
    YangDataDocument doc = YangkitUtils.loadValidYangDataDoc(context,
        "../data/yang-transformation/yang_transformation.json");
    ...
    container.addChild(leafNode);
    ...
    assertEquals("3", value);
}
```

Source: yangkit/src/test/java/YangTransformationTest.java

Table 11. Comparison of YANG-to-YANG transformation enablers

Transformation case	libyang	Yangkit	ODL Yang Tools
Add node defined in schema but absent from data	Supported.	Supported.	Supported.
Update node defined in schema and already present in data	Supported through data-tree modification APIs.	Partially supported. The evaluation results indicate that this does not work in the current behavior, although it could be changed in code.	Supported.
Add node not defined in schema	Not supported as a valid model-aware operation.	Not supported.	Not supported.

10. YANG Library Loading

The YANG Library feature concerns the ability to load a list of modules and enable features based on YANG Library data. This capability is relevant when a schema context must be reconstructed from a producer-provided module inventory rather than from a manually curated list of files.

The libyang code path for YANG Library loading creates a new context, iterates over the data tree, extracts module names, revisions and features, loads the corresponding modules and compiles the context. For Yangkit

and ODL Yang Tools, the repository-based tests indicate that code can be written to load a YANG Library. Yangkit is marked as supported in this area.

Table 12. Comparison of YANG Library support

Feature	libyang	Yangkit	ODL Yang Tools
Load module list from YANG Library	Supported through existing code in the test results.	Supported through code written for the workflow.	Supported through code written for the workflow.
Enable features from YANG Library	Supported by extracting feature values and loading modules with features.	Supported in the implemented workflow.	Supported in the implemented workflow.
Context construction	Creates and compiles a new libyang context.	Builds a Yangkit schema context from the library data.	Builds an ODL Yang Tools context from the library data.

11. Yang Push, Telemetry and Notification Envelope Validation

11.1 Yang Push Validation

Yang Push validation evaluates whether a library can consume and validate messages associated with a Yang Push producer. The test results indicate that libyang validates notification messages and that `yanglint` supports a notification-specific validation mode. Yangkit validates several Yang Push messages, including push-update and subscription-started examples. ODL Yang Tools cannot be tested in this area because structures are not supported in the required way.

```
@Test
void test1ValidPushUpdate() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadValidYangDataDoc("../yang/yangpush",
    "../data/yangpush/1-push-update.json");
}

@Test
void test1ValidSubscriptionStarted() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadValidYangDataDoc("../yang/yangpush",
    "../data/yangpush/1-subscription-started.json");
}

@Test
void test2ValidPushUpdate() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadValidYangDataDoc("../yang/yangpush",
    "../data/yangpush/2-push-update.json");
}

@Test
void test2ValidSubscriptionStarted() throws DocumentException, IOException, YangParserException {
    YangkitUtils.loadValidYangDataDoc("../yang/yangpush",
    "../data/yangpush/2-subscription-started.json");
}
```

Source: yangkit/src/test/java/YangPushTest.java

11.2 Telemetry Message Validation

Telemetry validation extends the analysis to messages that depend on multiple YANG modules and their dependencies. The libyang command-line test validates a Netgauze telemetry message using several telemetry-related modules. Yangkit is also marked as validating telemetry messages, with the requirement that all schemas and dependencies be loaded. ODL Yang Tools cannot be tested because the scenario depends on structure support.


```
yanglint -A -e \
yang/telemetry/ietf-telemetry-message@2025-10-19.yang
yang/telemetry/ietf-udp-notif-transport@2025-02-14.yang \
yang/telemetry/ietf-subscribed-notifications@2019-09-09.yang \
yang/telemetry/ietf-udp-notif-transport@2025-02-14.yang \
yang/telemetry/ietf-yang-push-telemetry-message@2025-10-19.yang
yang/telemetry/ietf-datastores@2018-02-14.yang \
-f json data/telemetry/valid-telemetry-msg-netgauze.json
```

Source: libyang / yanglint telemetry-validation command.

11.3 Notification Envelope Validation

Notification envelope validation is another structure-dependent scenario. The test results indicate that libyang validates notification envelope messages and can validate a Yang Push notification as a top-level node, as a child, or even when contained in anydata. Yangkit validates notification envelope messages. ODL Yang Tools is not applicable in the tested context because structure support is missing.

Table 13. Comparison of operational message validation scenarios

Operational scenario	libyang	Yangkit	ODL Yang Tools
Yang Push messages	Supported. Notification messages are validated, including with yanglint notification-specific mode.	Supported. Push-update and subscription-started examples are validated when schemas and dependencies are loaded.	Not applicable because structure support is missing for the tested scenario.
Telemetry messages	Supported through yanglint with the required telemetry modules.	Supported when all schemas and dependencies are loaded.	Not applicable because structure support is missing.
Notification envelopes	Supported. A Yang Push notification can be validated at different nesting levels, including within anydata.	Supported. Notification envelope messages are validated.	Not applicable because structure support is missing.
Analytical impact	Operational message validation is covered for the tested scenarios.	Operational message validation is covered, with schema/dependency completeness as a prerequisite.	Advanced message validation is blocked by the structure-support gap.

12. Schema Comparison

Schema comparison is treated as a feasibility-oriented feature. The evaluation refers to the draft `ietf-netmod-yang-schema-comparison` and describes whether the libraries can support tooling that compares two schema revisions.

For libyang, the evaluation results indicate that the `yanglint` tool, in a separate Git branch named `schema\_comparison`, is able to generate `ietf-yang-schema-comparison` YANG data for two revisions of a YANG module. For ODL Yang Tools, the feature is described as implementable because the library has the functions needed to implement the draft. For Yangkit, the feature is also described as implementable, and the repository includes code related to schema comparison.

Table 14. Comparison of schema-comparison support and feasibility

Feature	libyang	Yangkit	ODL Yang Tools
Schema comparison tooling	Supported through a separate `schema_comparison` branch of yanglint according to the test results.	Implementable. The evaluation results mention existing code for a schema comparison feature.	Implementable. The evaluation results indicate that the required functions are available.

Feature	libyang	Yangkit	ODL Yang Tools
Analytical impact	Most concrete implementation status among the three in the test results.	Promising but described as implementation-oriented rather than fully characterized.	Promising but described as implementation-oriented rather than fully characterized.

13. Cross-Feature Synthesis

Across the evaluated features, the comparison shows that the libraries do not differ only in the number of supported capabilities. They also differ in the nature of their support. Some features are available natively, others are achievable by composing lower-level APIs, and others are blocked by missing support for a prerequisite construct such as structures or schema-backed anydata.

The most recurrent pattern is that core schema and JSON data validation are broadly available across the three libraries, while advanced scenarios reveal sharper differences. Anydata validation, structure support, XPath behavior behind anydata, and telemetry or notification-envelope validation are the areas where the gap analysis becomes most informative.

Table 15. Global synthesis of support levels across evaluated feature families

Feature family	libyang	Yangkit	ODL Yang Tools
Schema loading and validation	Supported	Supported	Supported
Module inspection	Partially supported	Supported	Supported
JSON data validation	Supported	Supported	Supported
CBOR data validation	Not supported	Supported	Not supported
Anydata validation	Supported	Partially supported	Partially supported
YANG structures	Supported	Supported	Not supported
Anydata inside structures	Supported	Partially supported	Not applicable
Structures inside anydata	Supported	Supported	Not applicable
XPath/data extraction	Supported	Partially supported	Partially supported
Subtree validation	Supported	Supported	Supported
YANG-to-YANG transformation	Supported	Supported for the described add/update cases	Supported for the described add/update cases
YANG Library loading	Supported	Supported	Supported
Yang Push validation	Supported	Supported	Not applicable
Telemetry validation	Supported	Supported	Not applicable
Notification envelope validation	Supported	Supported	Not applicable
Schema comparison	Supported through specific yanglint branch	Implementable	Implementable

This synthesis should be read feature by feature. A “Supported” entry does not necessarily mean that the APIs are identical or that integration effort is equivalent. Conversely, a “Partially supported” entry may still be sufficient for specific use cases if the unsupported subcase is not required. The value of the matrix is to show where further technical attention is needed when selecting or integrating a library for a specific YANG workflow.

13.1 Interpretation by Workflow Type

A second way to read the results is to group features by workflow rather than by individual capability. Schema-centric workflows mainly require module loading, module validation and module inspection. Data-validation workflows require reliable association between a data instance and its schema, followed by full enforcement of structural and semantic constraints. Telemetry-oriented workflows add another layer of

complexity because they rely on structures, anydata, notification envelopes and complete dependency loading. Transformation workflows require direct manipulation of model-aware data trees.

For schema-centric workflows, the three libraries provide a usable basis, although libyang remains more context-oriented for inspection, while Yangkit and ODL Yang Tools expose more direct module-inspection paths. For data-validation workflows, all three libraries support JSON validation, but the exact validation sequence matters. The presence of additional validation steps for mandatory and must constraints means that a complete validation workflow must be distinguished from simple parsing.

For advanced payload workflows, anydata and structures become the decisive constructs. libyang and Yangkit cover the structure-dependent scenarios observed in the tests, while ODL Yang Tools cannot be evaluated for these scenarios because structure parsing is missing in the tested state. This affects not only structures themselves, but also anydata inside structures, structures inside anydata, Yang Push validation, telemetry validation and notification envelope validation.

For transformation workflows, the analysis shows that the ability to add a schema-defined missing node is available in both Java libraries, while update behavior differs between Yangkit and ODL Yang Tools. libyang is described in broader terms, with functions for creating, modifying, reconnecting and removing data-tree nodes. This difference matters when the expected workflow is not limited to validation but includes data normalization or data rewriting.

Table 16. Workflow-oriented interpretation of the comparison

Workflow type	Main required capabilities	Most relevant observed differences
Schema-centric analysis	Module loading, validation and inspection.	All three libraries can load and validate modules; standalone inspection is more direct in Yangkit and ODL Yang Tools than in libyang.
Data-instance validation	JSON parsing, schema matching, type checking, mandatory and must validation.	All three libraries support JSON validation; CBOR support differs significantly across the evaluated workflows.
Advanced payload validation	Anydata, schema-backed payload resolution, structures and anydata-structure interaction.	libyang and Yangkit cover more advanced scenarios; ODL Yang Tools is constrained by anydata and structure behavior.
Telemetry and notifications	Yang Push, telemetry messages, notification envelopes and dependency-complete schema contexts.	libyang and Yangkit validate the tested scenarios; ODL Yang Tools cannot be tested because structures are not supported.
Data transformation	Model-aware tree manipulation without intermediate serialization.	libyang exposes broader tree-manipulation capabilities; ODL Yang Tools supports add/update cases; Yangkit supports additions but not the tested update behavior.

14. Conclusion

The feature-based gap analysis shows that libyang, Yangkit and ODL Yang Tools all cover important parts of the YANG processing workflow, especially schema loading and JSON data validation. However, the detailed comparison also shows that advanced features create meaningful differences between the libraries.

libyang is consistently observed as supporting a broad set of advanced validation and data-tree manipulation scenarios, including strict anydata validation, structures, XPath-related extraction, subtree validation, YANG-to-YANG transformation, Yang Push, telemetry, notification envelope validation and schema comparison through a dedicated yanglint branch.

Yangkit provides substantial coverage for the evaluated Java-based scenarios. It supports schema validation, JSON validation, self-contained CBOR validation through Jackson conversion, structures, structure-in-anydata validation, subtree validation, YANG Library loading, Yang Push, telemetry, notification envelope validation and

schema comparison feasibility. Its main observed differences are linked to anydata strictness, some XPath extraction cases and the current behavior of update operations during YANG-to-YANG transformation.

ODL Yang Tools supports several core features, including schema validation, module inspection, JSON validation, schema extraction, subtree validation, YANG Library loading and some transformation scenarios. The most significant analytical constraint is the absence of usable structure parsing in the tested state. This prevents several advanced scenarios from being evaluated, including structure-dependent anydata workflows, telemetry messages, Yang Push messages and notification envelopes.